

Group 5 — Finance App (Finnhub)

Development Plan · v4

- Team** Sergio Rodriguez (Team Lead) + Carlos Rivera
- Concept** Stock Watchlist / Market Explorer
- Stack** Vite + React + TypeScript, React Router, Axios, localStorage, Git/GitHub
- Auth** Hardcoded credentials in `src/config`, form + validation, localStorage session, protected routes — no backend.

Daily milestones

Day	Date	Goal	Owner
1	Thu 8/13	Kickoff, theme, Finnhub key signup, Vite scaffold, git init, first commits	Both
2	Fri 8/14	Email theme for approval (deadline) ; React+TS learning sprint; wireframe + color scheme	Both
3	Mon 8/17	Routing shell + Navbar/Footer layout, CSS design system, hooks learning	Carlos
4	Tue 8/18	<code>finnhubService.ts</code> (Axios) + types; verify search/quote/profile/news endpoints; <code>.env</code> handling	Sergio
5	Wed 8/19	Auth (hardcoded) : config credentials, login form + validation, AuthContext, localStorage session, protected routes (~half day)	Carlos
6	Thu 8/20	Market page : browse US stocks, search, sector filter, sort by change, loading/error/empty states	Sergio
7	Fri 8/21	Stock detail page (<code>/stock/:symbol</code>): quote, profile, latest news, peers	Carlos
8	Mon 8/24	Watchlist : add/remove buttons, protected Watchlist page, context + localStorage persistence	Sergio
9	Tue 8/25	Responsive pass (mobile-first), polish, form validation feedback, consistent empty/error states	Both
10	Wed 8/26	All 15 user stories pass — manual QA, bug fixes, edge cases (429, offline, empty watchlist)	Both
11	Thu 8/27	README (setup, architecture, API docs, features, limitations), code cleanup	Both
12	Fri 8/28	Presentation prep: demo script, speaking roles, first dry run	Both
—	Sat/Sun 8/29–30	Final dry runs; CODE FREEZE Sun 8/30	Both
—	Mon 8/31	Presentation	Both

Standing responsibilities

- **Sergio (Lead)**: Finnhub service layer + types, Market page, Watchlist, PR reviews, daily standup updates to trainer
- **Carlos**: routing shell + layout, auth (hardcoded), Stock detail page, responsive/polish
- **Shared**: scaffold, React/TS learning, watchlist QA, responsive, QA, README, presentation

Feature work is alternated day-by-day so both members own a roughly equal mix of API and frontend work.

User story requirements (15)

ID	Requirement	Planned
AUTHENTICATION		
☐ US-01	Log into the application to access protected features	Day 5
☐ US-02	Log out so the session ends securely	Day 5
☐ US-03	Login state persists while continuing to use the application	Day 5
NAVIGATION		
☐ US-04	Navigate between pages without refreshing the browser	Day 3
☐ US-05	Protected pages require authentication before they can be viewed	Day 5
EXTERNAL API		
☐ US-06	View information retrieved from an external API	Day 4-6
☐ US-07	Application indicates when data is loading	Day 6
☐ US-08	Meaningful feedback if the API cannot be reached	Day 6, 10
SEARCH AND FILTERING		
☐ US-09	Search available data	Day 6
☐ US-10	Filter the displayed results	Day 6
☐ US-11	View detailed information about an individual item	Day 7
USER EXPERIENCE		
☐ US-12	Information displayed in a clean and organized layout	Day 3, 9
☐ US-13	Forms validate input before submission	Day 5, 9
☐ US-14	Application works across different screen sizes	Day 9
☐ US-15	Meaningful error messages whenever something goes wrong	Day 9, 10

All 15 must pass manual QA on Day 10 (Wed 8/26).

Project requirements checklist (10)

☐	Built as a React application using modern React practices and a clean project structure
☐	Uses reusable components, React Hooks, and client-side routing to organize functionality
☐	Integrates with at least one external REST API, handling loading, success, and error states
☐	Functional login/logout experience with protected routes and persistent auth state
☐	Intuitive navigation, responsive layouts, and a polished user experience
☐	Validates user input and provides clear feedback for invalid input and application errors
☐	Clean, maintainable, well-documented code following JavaScript and React best practices
☐	Appropriate testing or validation of key application functionality
☐	Complete README with setup instructions, architecture, API documentation, and execution steps
☐	Shared Git repository with meaningful commit history showing consistent contributions from both members

Notes

- No backend work — frontend-only; API integration lives in the service layer
- Free tier 60 calls/min → cache quote/profile data; fetch per-symbol only on detail load; no charts (/stock/candle is 403-prone on free keys)
- API key in `.env` (gitignored) + committed `.env.example`
- Market News page is optional stretch if ahead of schedule